COMP3000 Proposal (VidBriefs (2.0))

[Youtube Video] + [User Question] -> [Jev; learn from SQLite history] -> [AI Panel that ends with master critique; may be more levels?]

1. Overview

VidBriefs (2.0); an innovative idea created to be unprecedented in how users interact with and consume video content. Essentially, it is made for people who wish to consume the video's insights efficiently without having to watch the entire video. I will leverage advanced Deep Learning techniques, in a high-level way whilst utilising APIs, to ensure that the logged-in user receives personalised responses tailored to their viewing habits and preferences.

1.1 Idea and Why

Early post injury, I found it hard to concentrate, particularly on things that I don't necessarily care about (engage into), e.g., *why am I even intrigued to watch this in the first place?* What did I want to learn or gain from the video? I therefore created *VidBriefs* wherein I utilised the `youtube-transcript-api` on Python to extract video transcripts from their respective YouTube video link, then fed it into `gpt-3.5-turbo` or `gpt-4` in a looped conversation wherein the AI will answer specific questions and provide concise summaries based on my interests and queries.

I will attempt to find new, light scientific discoveries regarding information-consumption via AI

1.2. What Is Different

1.2.1 VidBriefs 1.0

This was made as an iOS app which initially would just extract the youtube video transcript from the link, feed it to AI, then allow the user to prompt the AI with specific

questions regarding it.

However, this was created primarily for my personal use where I did not delve deeper as the main idea was finalised in my eyes: if I want to learn something from a YouTube video without watching the whole thing, AI can just read the transcript in seconds and provide me with the key insights concerning what I specifically asked about (not just a general summary).

1.2.2 VidBriefs 2.0

VidBriefs (2.0) takes this innovative concept further by incorporating more advanced AI techniques: 1) learn user preferences and viewing habits over time to provide increasingly personalised summaries of not just certain videos/channels, but also ***chapters*** of the video within the video (this could perhaps utilise youtube's own chapters along with the transcript-content of each video combined), 2) utilise more sophisticated natural language processing models to generate higher-quality insights, 3) answer a user's question by pointing to the exact timestamped parts of the video that answer it, 4) learn from how the user treats each extracted part (played through, skipped, replayed) instead of only asking them to rate anything (with a slider which could be hidden into another interface element so the user doesn't feel like they are manually rating), 5) use the video itself and not only the transcript, so things shown but not said are captured, 6) generate a few questions at the end, both to check what they learnt and to learn what they care about 7) create the video-chat with an overview of the questions etc

1.2.3 Improvements

Essentially, second-generation VidBriefs now ***intends*** to be structured as a Retrieval-Augmented Generation (RAG) system with re-ranking, wherein the AI continuously learns the user's focus i.e. the user's interactions with the video content to improve its prioritisation, i.e. VidBriefs' relevance and correctness of the generated summaries; the decision-making behaviour will need to be robustly validated on-the-fly subtly concerning the user's evolving interests relative to the video content. (basically: 1) find the relevant parts first, 2) put the best ones on top for the user, 3) let the AI respond using only those parts, 4) keep learning from what the user does and keep checking it is still getting it right as their interests transition.)

1.3. Methodology

- also intend to utilise Google Cloud Developer Plugin but not sure how yet

1. Extract video transcript using youtube-transcript-api, faster-whisper as a fallback where captions are disable), as well as visual content via gemini-3.8-flash, and the 'most replayed' heatmap via yt-dlp where the video has one.
    - then split the video into parts with timestamps, using YouTube's own chapters where they exist, otherwise chunks of the transcript (fixed windows of roughly 60-90 seconds), wherein each part is given its own ID

2. Feed the video's content:
    - Jev: the first 'system-one' AI that won't generate text, but will be utilised as an "invisible engine" which supports the master AI's understanding, thus overall response via weighted relevance scores.
        - **Input**: the parts of the video plus what this person asked before and if they played, skipped or replayed anything, i.e. for each part: 1) how similar it is to the current question, 2) how similar it is to the user's previous questions, 3) the user's play, skip and replay history on similar parts, 4) its 'most replayed' value, 5) its position and length in the video (similarity via an off-the-shelf sentence encoder from Sentence-Transformers)
        - **Output**: a relevance-score for each part of the video scored on its importance aligned with the user's interests and previous interactions, what the AI thinks too and perhaps which of my prior questions it is related to included in the ranking model, wherein the highest-scored parts are passed on
        - **How**: Jev is a LightGBM ranker (off-the-shelf), thus the one new component is what it learns from, not a new architecture. The maths: each part i has the inputs above as x_i, and Jev learns score_i = f(x_i), trained with LambdaRank so that parts the user played rank above parts they skipped
        - **Cold start**: before a user has any history, Jev falls back to question similarity plus the 'most replayed' value; this also gives the one generic version for everyone (the 'absolute video'), whilst the personalised version takes over as the user's history grows
    - Reading-AI or reading-panel: the AI(s) that directly get and consider the video(s) content (utilise gemini-3.8-flash if video is to be visually analysed too).

    - Master AI: an existing model used via its API which reads the parts Jev scored highest, then what the reading-AI or reading-panel deemed most relevant, and generates the final summary that answers the user. It returns only the IDs of the parts it used (as structured JSON); the app checks each ID exists and attaches that part's timestamp, so the master AI never writes a timestamp itself and cannot invent one.

3. Save what the user does: the YouTube IFrame Player API records whether each part is played through, skipped or replayed, which is stored in SQLite and used to retrain Jev next time.

4. Evaluation: compare 1) the parts in chapter order with no ranking, 2) ranking by similarity to the question only, 3) Jev; measured with NDCG@5 (how often the parts the user actually played are in the top five) on later sessions held out from training, plus a short human rating of the final summaries. Data comes from my own use from the start, plus a small study of 5-10 people over a few weeks (subject to ethics approval). Gemini's use within the app will be stated in the AI declaration.

- ☐ high-level diagram